Илья Романович

Backend & Embedded Systems Engineer

Минск, Беларусь · ilya.business.contact77@gmail.com · github.com/mdedz · LinkedIn

О себе

Разрабатываю системы «ниже» и «выше» API одинаково уверенно: от протоколов последовательной связи и embedded-прошивок до backend-сервисов и веб-интерфейсов. Спроектировал и реализовал платформу телеметрии в реальном времени для игровых автоматов: embedded C (ESP32/FreeRTOS), backend на Python/FastAPI и панель оператора на React — включая протокол идентификации устройств на Ed25519, независимо реализованный на Python и на embedded C. Отдельно построил и поддерживал платформу управления парком Android-TV устройств с низкоуровневым ADB/GPIO-взаимодействием и сервером на Django/Channels с подписанным WebSocket RPC и безопасными OTA-обновлениями. Комфортно веду систему целиком: от протокола до продакшена.

Технические навыки

- **Backend:** Python, FastAPI, Django, Django REST Framework, SQLAlchemy, Alembic, PostgreSQL, Redis, Celery
- **Системное и embedded-программирование:** C, C++, Rust, ESP-IDF/FreeRTOS, последовательные протоколы UART/RS-232, CRC16, GPIO, ADB, systemd
- **Безопасность и аутентификация:** JWT, OAuth2, Ed25519 / прикладная криптография, HMAC-подписанные сообщения, TLS
- **Frontend:** React 19, TypeScript, Vite, Redux Toolkit, TanStack Query, Tailwind CSS
- **Инфраструктура:** Docker, Docker Compose, GitHub Actions CI, Linux, Nginx
- **Интеграция AI/LLM:** OpenAI / Azure OpenAI / Groq SDK, function/tool-calling, валидация структурированного вывода, rate limiting

Опыт работы

Независимый инженер-разработчик **2020 – настоящее время**
Самозанятость, удалённо

Проектирую, разрабатываю и сопровождаю backend- и embedded-системы для телеметрии игорной индустрии, управления парками Android-TV устройств и прикладной автоматизации на базе LLM — от постановки протокола до развёртывания, напрямую с заказчиками.

- Спроектировал multi-tenant backend на FastAPI и протокол zero-touch регистрации устройств (proof-of-possession на Ed25519, короткоживущие JWT для машинной идентификации) для платформы телеметрии игровых автоматов, затем независимо переписал прошивку коллектора на embedded C под ESP32, подтвердив, что протокол действительно не привязан к конкретному «железу».
- Реверс-инжинирил специфику управления Android-TV на уровне ADB (пути input-сервисов по чипсетам, тайминги DPAD-навигации), реализовал аппаратный резерв включения питания через GPIO-реле и self-healing watchdog для удалённо управляемого парка устройств, в паре с сервером на Django/Channels с подписанным WebSocket RPC и безопасными OTA-обновлениями через GitHub App.
- Построил систему компьютерного зрения в реальном времени (многопоточный C++, инференс на NPU) с двумя независимыми статистическими валидаторами калибровки и дрейфа — для выявления незаметного смещения камеры/освещения, которое иначе молча искажало бы результат.
- Реализовал протестированные и покрытые CI backend-сервисы на Django/DRF (система бронирования, кэширующая интеграция с rate limiting) на уровне тестового задания, а также self-hosted IaaS-платформу (Rust-агент гипервизора + Django backend + React frontend) как полноценное full-stack упражнение.
- Разработал CLI-инструмент автоматизации с реальной инженерией надёжности LLM: детекцией и восстановлением после обрезанных ответов модели, fail-open обработкой неоднозначной классификации, провайдер-независимой интеграцией LLM — с CI и полным pipeline тестов/линтеров.

Ключевые проекты

Платформа телеметрии игровых автоматов (протокол SAS) *Python (FastAPI) · embedded C (ESP32/FreeRTOS) · TypeScript/React · PostgreSQL*

Спроектировал multi-tenant backend и панель оператора для телеметрии игровых автоматов в реальном времени с кросс-языковым протоколом идентификации устройств на Ed25519 (proof-of-possession регистрация, короткоживущие машинные JWT), независимо реализованным на Python и вручную портированным на embedded C. Решил задачу эмуляции 9-битного wakeup-сигнала UART поверх 8-битного аппаратного UART через переключение чётности MARK/SPACE. Покрыто набором тестов из 16 файлов (3000+ строк), включая полный end-to-end сценарий.

Платформа управления парком Android-TV устройств *Python (Django/Channels/ASGI) · Flask · WebSockets*

Построил платформу удалённого управления Android-TV устройствами: подписанный, коррелированный WebSocket RPC-протокол, соединяющий асинхронную модель Django Channels с синхронными REST-обработчиками; безопасный механизм OTA-обновлений на коротко живущих токенах установки GitHub App; агент устройства с реальным реверс-инжинирингом ADB/GPIO и многоуровневым восстановлением после сбоев (программный retry – аппаратное реле – перезагрузка хоста через watchdog).

Система компьютерного зрения в реальном времени *C++ · OpenCV · RKNN (NPU)*

Построил многопоточный, ускоренный на NPU конвейер трекинга объектов в реальном времени с двумя независимо спроектированными статистическими валидаторами – один контролирует качество калибровки, второй детектирует дрейф во время работы через корректную циркулярную статистику и state machine с гистерезисом.

Self-hosted IaaS control plane *Rust (Axum/bollard) · Django/DRF · React/Redux · Docker*

Построил full-stack self-hosted платформу управления инфраструктурой: агент-гипервизор на Rust с реальным управлением жизненным циклом дисков через loop-устройства (создание/форматирование/монтирование/resize ext4-образов) и оркестрацией Docker-контейнеров, backend на Django/Celery/Redis с multi-tenancy на основе квот, и frontend на React/Redux с live-опросом использования ресурсов.

Автоматизация поиска работы на базе AI *Python · Groq/OpenAI-совместимые LLM API*

Построил покрытый CI CLI-инструмент для автоматизации взаимодействия с job-платформой с реальной инженерией надёжности LLM: рабочий протокол детекции и продолжения обрезанных ответов, fail-open обработку неоднозначного вывода классификации модели, провайдер-независимую абстракцию LLM – полный набор линтеров/тайп-чекеров (ruff, mypy `--strict`, pre-commit) и CI на GitHub Actions.

Backend-сервисы на Django REST Framework *Python · Django REST Framework · PostgreSQL · Redis*

Реализовал два backend-сервиса на уровне тестового задания, покрытых CI: систему бронирования с корректной валидацией пересечения интервалов и кэширующую интеграцию с внешним API с rate limiting и health-check эндпоинтом – оба полностью протестированы (9+ тестовых файлов каждый) и разворачиваются через Docker.

Образование

Белорусская государственная академия связи (БГАС) – обучение не завершено